

PROJECT VIKSIT BHARAT 2026

AccessMitra

Project Documentation — Technical Reference

Problem Statement :

"A City Full of Barriers": Accessibility & Inclusion

Team Name :

SheCodes

Document Version 1.0

Contents

1. Overview
2. Installation & Setup Instructions
3. Project Structure
4. System Architecture & Workflow
5. Technology Stack & Dependencies
6. API Details
7. Database Schema
8. Deployment Details
9. Technical Implementation Details
10. Known Limitations

1. Overview

AccessMitra is a community-verified accessibility navigator. It helps people find step-free routes and accessibility-verified places — ramps, accessible restrooms, lifts, and parking — report barriers with photo evidence, and have those reports confirmed by other users before the map updates. It applies a Waze-style trust model to physical accessibility.

This document describes the Stage 2 prototype: a working REST API backed by PostgreSQL/PostGIS, a React Native (Expo) client that consumes that API, and a self-contained HTML reference build used for rapid UX demonstration. It is a separate technical artifact from the Stage 1 pitch deck and focuses entirely on how the system is actually built and how to run it.

What's Implemented and Running

- Full Express REST API — places, routing, reports, upvote/verification, leaderboard, auth
- PostgreSQL schema with PostGIS geospatial queries (radius search, distance sort)
- Community verification state machine (pending → verified) with live feature-flag updates
- React Native (Expo) client — 5 screens, real navigation, calls the live API
- Self-contained interactive HTML demo with the same UX and mock data

What's Stubbed, With a Clear Integration Point

- Google Maps Directions — falls back to a deterministic mock route
- CV-based photo verification (TensorFlow/OpenCV) — stub returns a placeholder result
- Firebase Cloud Messaging push notifications — logs to console instead of sending
- Photo upload to S3 — accepts a URL field; upload pipeline not yet wired

2. Installation & Setup Instructions

2.1 Prerequisites

- Node.js 18+ and npm
- Docker & Docker Compose (recommended — avoids installing Postgres/PostGIS locally)
- Expo Go app on a phone, or an Android/iOS simulator, for on-device testing of the mobile client

2.2 Option A — Full Stack with Docker (fastest, recommended)

```
git clone <repository-url> accessmitra
cd accessmitra
docker compose up --build

# in a second terminal, once containers are healthy:
docker compose exec api npm run seed

# verify:
```

```
curl http://localhost:4000/api/health
curl http://localhost:4000/api/places
```

This starts a PostgreSQL 16 + PostGIS container (schema auto-applied on first boot from backend/src/db/schema.sql) and the Express API on port 4000.

2.3 Option B — Manual Local Setup

```
cd backend
npm install
cp .env.example .env # edit DATABASE_URL to point at your local Postgres
npm run seed
npm run dev           # starts on http://localhost:4000
```

2.4 Running the Mobile Client

```
cd mobile
npm install
npm run web      # opens the app in a browser via react-native-web (fastest way to try it)
# or:
npm run android # requires Android Studio / an emulator
npm run ios      # requires Xcode / iOS simulator (macOS only)
```

Set EXPO_PUBLIC_API_URL in mobile/.env if the API isn't on localhost:4000 — for example, when testing on a physical device on the same Wi-Fi network.

2.5 Zero-Install Demo

Open demo/index.html directly in any browser. No install, no backend — useful for a first look at the UX before setting up the full stack.

Sandbox Verification Note

This authoring environment has no outbound access to the npm or Docker Hub registries, so the commands above could not be executed end-to-end here. Every backend file was syntax-checked (node --check across all JavaScript) and the SQL was reviewed against the PostGIS 3.4 function reference. All commands are written to run as-is with a normal internet connection.

3. Project Structure

```
accessmitra/
├── README.md
├── docker-compose.yml
├── demo/
│   └── index.html
```

Self-contained interactive UX reference

```

├── backend/                                Node.js + Express REST API
│   ├── package.json
│   ├── Dockerfile
│   ├── .env.example
│   ├── seed/seed.js                       Demo data loader
│   └── src/
│       ├── server.js                      App entry point / API gateway
│       ├── db/
│       │   ├── schema.sql                PostgreSQL + PostGIS schema
│       │   └── pool.js                   pg connection pool
│       ├── middleware/
│       │   ├── auth.js                   JWT verification
│       │   └── rateLimiter.js            Fixed-window rate limiter
│       ├── routes/                       Express routers (one per resource)
│       │   ├── places.js reports.js routes.js community.js auth.js
│       ├── controllers/                  Request handling + SQL queries
│       │   ├── placesController.js reportsController.js
│       │   ├── communityController.js authController.js
│       └── services/                     External-integration boundaries
│           ├── routingService.js          (Google Directions)
│           ├── aiVerificationService.js   (CV model)
│           └── notificationService.js     (Firebase Cloud Messaging)
├── mobile/                                React Native (Expo) client
│   ├── App.js
│   ├── app.json
│   ├── package.json
│   └── src/
│       ├── navigation/AppNavigator.js    Bottom tabs + nested stack
│       ├── screens/                      Home, Routes, Report, Community, Profile
│       ├── components/PlaceCard.js
│       ├── services/api.js               fetch wrapper around the backend
│       └── theme/colors.js               Shared design tokens

```

3.1 Why This Layout

The backend is organised as routes → controllers → services: routes only wire URLs to handlers, controllers hold request/response and SQL logic, and services isolate every external integration (maps, AI, push notifications) behind a single function so those can be swapped from mock to real without touching any controller. The mobile app mirrors the five product surfaces — Home, Routes, Report, Community, Profile — 1:1 with the backend's resource boundaries (places / routes / reports / community), so each screen maps predictably to the API section in Chapter 6.

4. System Architecture & Workflow

4.1 Layered Architecture

Layer	Components
Client Layer	React Native app (iOS / Android / Web via Expo) · HTML reference demo
API Gateway Layer	Express app (server.js) · CORS · JSON body parsing · Rate limiter (120 req/min/IP) · JWT auth middleware
Application Services	Places Service · Routing Service · Reports / Trust Service · Community Service · Auth Service
External Integrations	Google Maps Directions (mocked) · CV verification service (stubbed) · Firebase Cloud Messaging (stubbed) · S3 photo storage (planned)
Data Layer	PostgreSQL 16 · PostGIS 3.4 (geography columns, radius queries)

4.2 Core Workflow: Report → Verify → Live Map Update

1. Discover. Client calls GET /api/places with the user's lat/lng; the Places Service runs a PostGIS ST_DWithin + ST_Distance query and returns nearby places sorted by distance, each carrying a features JSON object (ramp / restroom / elevator / parking / audio).
2. Navigate. Client calls GET /api/routes?placeId=. The Routing Service checks for the highest-upvoted pending report against that place and, if one exists, injects a live "barrier ahead — rerouting" step into the returned step list.
3. Report. The user submits POST /api/reports with a place, category, description, and optional photo URL. The AI Verification Service is called as a pre-screen (currently a stub) and the report is stored with status = 'pending'.
4. Community verification. Other users call POST /api/reports/:id/upvote. Inside a single DB transaction: the upvote count increments; at 3 confirmations the report flips to verified, the related place's features JSONB flag is set to false for that category (e.g. a verified "Broken Ramp" report clears ramp), the place's verified_count increments, and the reporter's trust_points increase.
5. Notify. On verification, the Notification Service alerts the original reporter (stubbed to console logging until Firebase credentials are configured).
6. Reflect. The next GET /api/places call from any client immediately reflects the updated features — there is no cache to invalidate, so "the map updates live" is a direct consequence of the transaction in step 4, not a separate sync step.

5. Technology Stack & Dependencies

5.1 Backend

Package	Version	Purpose
express	^4.19.2	HTTP server & routing
pg	^8.12.0	PostgreSQL client / connection pool
jsonwebtoken	^9.0.2	Auth token issuance & verification
bcryptjs	^2.4.3	Password hashing
cors	^2.8.5	Cross-origin requests from the mobile/web client

Package	Version	Purpose
morgan	^1.10.0	HTTP request logging
dotenv	^16.4.5	Environment variable loading

5.2 Mobile Client

Package	Version	Purpose
expo	~51.0.0	React Native tooling, build & dev server
react-native	0.74.5	Cross-platform UI runtime
@react-navigation/native + bottom-tabs + native-stack	^6.x	Tab navigation + nested route stack
react-native-screens / -safe-area-context	latest compatible	Navigation performance & safe-area handling

5.3 Data & Infrastructure

Component	Role
PostgreSQL 16 + PostGIS 3.4	Primary data store; geography columns power radius/distance queries
Docker Compose	Local orchestration of the database + API for one-command setup
Google Maps Directions API (planned)	Turn-by-turn walking directions; integration point in routingService.js
TensorFlow / OpenCV microservice (planned)	CV pre-screening of report photos; integration point in aiVerificationService.js
Firebase Cloud Messaging (planned)	Push notifications on verification; integration point in notificationService.js
AWS S3 (planned)	Report photo storage

This mirrors the stack named in the Stage 1 deck. Every "planned" row has working, isolated code in backend/src/services/ with a documented TODO showing exactly where the real API call goes — see Chapter 9.

6. API Details

Base URL: http://localhost:4000/api. All request/response bodies are JSON. Endpoints marked POST that create or mutate data accept an optional Authorization: Bearer <token> header — if omitted, the action is recorded anonymously.

6.1 Places

Endpoint	Description
GET /places	List places. Query params: lat, lng, radiusM (default 5000), category, need (wheelchair/visual/hearing/elderly), search. Sorted by distance when lat/lng given.
GET /places/:id	Full detail for one place, including its raw features JSON.

6.2 Routing

Endpoint	Description
GET /routes?placeId=&fromLat=&fromLng=	Returns a step-free route to the place: total distance, ETA, and an ordered step list. Automatically injects a reroute step if that place has an active pending barrier report.

6.3 Reports

Endpoint	Description
GET /reports?status=&placeId=	List barrier reports, newest first.
POST /reports	Body: { placeId, category, description, photoUrl }. Creates a report with status: "pending".
POST /reports/:id/upvote	Records one confirmation. At 3 upvotes: flips status to verified, clears the corresponding accessibility flag on the place, increments the place's verified count and (if authenticated) the reporter's trust points.

6.4 Community

Endpoint	Description
GET /community/leaderboard	Top contributors ranked by trust points, with a derived badge tier (New Contributor / Silver / Gold Verifier).

6.5 Auth

Endpoint	Description
POST /auth/register	Body: { name, email, password, accessibilityNeed }. Returns the created user and a JWT.
POST /auth/login	Body: { email, password }. Returns the user and a JWT (7-day expiry).

6.6 Example

```
curl
"http://localhost:4000/api/places?lat=30.7410&lng=76.7794&radiusM=3000&need=wheelchair"
```



```
{
  "results": [
    {
      "id": 1,
      "name": "Sector 17 Metro Station",
      "category": "Transit",
      "rating": 4.6,
      "verified_count": 128,
      "features": { "ramp": true, "restroom": true, "elevator": true, "parking": false,
"audio": true },
      "distance_m": 412.7
    }
  ]
}
```

7. Database Schema

PostgreSQL 16 with the postgis and pgcrypto extensions. Full DDL lives in backend/src/db/schema.sql; summarised below.

7.1 users

Column	Type	Notes
id	UUID	PK, gen_random_uuid()
name, email	TEXT	email UNIQUE
password_hash	TEXT	bcrypt
accessibility_need	TEXT	wheelchair / visual / hearing / elderly / null
trust_points	INTEGER	default 0; incremented on verified reports

7.2 places

Column	Type	Notes
id	SERIAL	PK
name, category, address	TEXT	category is CHECK-constrained to a fixed set
location	GEOGRAPHY(POINT,4326)	PostGIS column, GIST-indexed for fast radius search
rating	NUMERIC(2,1)	
verified_count	INTEGER	
features	JSONB	{ramp, restroom, elevator, parking, audio} booleans

7.3 reports

Column	Type	Notes
id	SERIAL	PK
place_id	INTEGER	FK → places, ON DELETE CASCADE
reporter_id	UUID	FK → users, ON DELETE SET NULL (nullable — anonymous reports allowed)
category	TEXT	CHECK-constrained (Broken Ramp, Blocked Path, ...)
description, photo_url	TEXT	photo_url populated by the (planned) upload pipeline
status	TEXT	pending / verified / rejected
upvotes	INTEGER	default 0; verification threshold is 3

7.4 report_votes

Composite primary key (report_id, user_id) — prevents a single user from upvoting the same report more than once. Referenced by application logic but not yet enforced at the controller level (see Chapter 10).

7.5 Entity Relationships

```
users    1—∞ reports      (reporter_id)
users    1—∞ report_votes (user_id)
places   1—∞ reports      (place_id)
reports  1—∞ report_votes (report_id)
```

7.6 Example Spatial Query

```
SELECT p.*, ST_Distance(location, ST_MakePoint($1,$2)::geography) AS distance_m
FROM places p
WHERE ST_DWithin(location, ST_MakePoint($1,$2)::geography, $3)
ORDER BY distance_m ASC;
```

8. Deployment Details

8.1 Current State (Stage 2)

The prototype is designed to run locally via Docker Compose (Section 2.2) or manually against any PostgreSQL/PostGIS instance. It has not been deployed to a public URL for this submission — judges run it locally per Section 2, or review the zero-install demo/index.html for the UX without any setup.

8.2 Intended Production Deployment

Component	Target
API (backend/)	Containerised (Dockerfile included) on AWS ECS / Render / Railway, behind a managed load balancer
Database	Managed PostgreSQL with PostGIS (AWS RDS or Supabase)
Photo storage	AWS S3 with a CDN in front (CloudFront)
Mobile app	Built via EAS Build (Expo) and distributed through the Play Store / App Store
CI/CD	GitHub Actions — lint + node --check on push, Docker image build on merge to main

8.3 Environment Configuration

All configuration is via environment variables (see backend/.env.example): DATABASE_URL, JWT_SECRET, and optional GOOGLE_MAPS_API_KEY / AI_SERVICE_URL / FIREBASE_SERVICE_ACCOUNT — each service in src/services/ checks for its own key and transparently falls back to a mock if it isn't set, so the same codebase runs in dev without any paid credentials.

9. Technical Implementation Details

9.1 Community Verification as a Single Transaction

The upvote → verify → update-place-features → award-points sequence in reportsController.upvoteReport runs inside one Postgres transaction with a SELECT ... FOR UPDATE row lock on the report. This prevents a race where two simultaneous upvotes both read upvotes = 2 and only one increments past the threshold — a real concern for a crowdsourced counter under concurrent load.

9.2 Geospatial Search via PostGIS, Not Application Code

Distance and radius filtering happen in SQL (ST_DWithin, ST_Distance on a GIST-indexed geography column) rather than fetching all rows and computing Haversine distance in JavaScript. This scales to a full city's worth of places without the app server becoming the bottleneck.

9.3 Service-Boundary Pattern for External Integrations

Every third-party dependency named in the Stage 1 stack — Maps, CV model, push notifications — sits behind a single async function in backend/src/services/ that checks for its own API key/URL environment variable and falls back to a clearly-labelled mock otherwise. This was a deliberate choice so that:

- the request/response shape the rest of the app depends on is finalised now, before any paid integration is wired up;
- the app is fully testable in CI/local dev with zero external credentials;
- going from stub to production integration for any one of them is a change inside a single file.

9.4 Accessibility Features Flip Automatically From Verified Reports

FEATURE_BY_CATEGORY in reportsController.js maps a verified report's category directly onto the affected place's features JSONB key (e.g. a verified "Broken Ramp" report sets features.ramp = false via jsonb_set). This is what makes "the map updates itself" a real, mechanical consequence of community verification, not a manual moderation step.

9.5 Mobile Client Architecture

Screens are grouped under a bottom tab navigator; the Routes tab additionally nests a stack navigator (RouteList → RouteDetail) so selecting a destination pushes a detail screen with its own back button, matching standard mobile navigation conventions rather than flattening everything into tabs. All network access goes through a single src/services/api.js module so the base URL, headers, and error handling are defined once.

9.6 Design Tokens Shared Across Every Surface

Colours (navy / teal / amber civic-signage palette) and the spacing scale are defined once in mobile/src/theme/colors.js and mirrored as CSS custom properties in demo/index.html, so the pitch deck, the instant demo, and the real app are visually the same product rather than three different-looking prototypes.

10. Known Limitations

Area	Limitation	Planned Resolution
Duplicate upvotes	report_votes table exists in the schema but the upvote endpoint does not yet check it, so the same unauthenticated client can upvote a report more than once.	Enforce via the composite primary key on report_votes and reject duplicate inserts in upvoteReport.
Photo upload	Reports accept a photoUrl string but there is no upload endpoint yet — photos must be hosted externally and linked.	Add a pre-signed S3 upload endpoint; wire photo_url to the returned object URL.
Routing realism	Without a configured GOOGLE_MAPS_API_KEY, routes are a deterministic mock, not a real walking path.	Wire the documented TODO in routingService.js to Google Directions / OSRM.
AI verification	No CV model is trained or deployed; aiVerificationService.js is a stub.	Train a lightweight ramp/obstruction classifier and deploy as a small FastAPI service behind AI_SERVICE_URL.
Push notifications	Verification notifications are logged to the server console, not delivered to a device.	Wire the Firebase Admin SDK once a service account is provisioned; store device tokens per user.
Auth on mobile	The mobile client calls the API anonymously by default; login/register screens exist on the backend but are not yet wired into the mobile UI.	Add a Profile-tab login flow that stores the JWT and attaches it to write requests.
Offline support	No offline cache — the app requires connectivity to load places/reports.	Cache the last-fetched place list locally for degraded-connectivity use, common in exactly the transit contexts this app targets.

Area	Limitation	Planned Resolution
Test coverage	No automated test suite yet; correctness was checked via manual API testing and code review, not CI tests.	Add integration tests for the controllers (especially the upvote transaction) using a test database.
Sandbox verification	This authoring environment could not reach the npm/Docker registries, so npm install / docker compose up were not executed end-to-end here — only syntax-checked.	N/A — resolved simply by running the documented setup with normal internet access.

— End of Document —